

O Codificador Limpo:

Um Código de Conduta para Programadores Profissionais



Capítulo 5: Desenvolvimento Guiado por Testes (TDD)



PET/ADS/IFSP

**Este material foi desenvolvido pelo grupo
PET/ADS do IFSP São Carlos**

O Veredito e a Defesa

O debate sobre a eficácia do TDD está oficialmente **encerrado** .

- A indústria já superou a discussão se a técnica funciona ou não.

Cirurgiões **não precisam** defender a lavagem das mãos.

- A prática de testes constitui a higiene básica e inegociável do código.

As Três Leis do TDD (Regras Fundamentais)

1ª Lei: Você **não** escreve código de produção sem um teste que falhe.

- O desenvolvimento inicia **obrigatoriamente** pela criação de uma prova de erro.

2ª Lei: Você escreve **apenas** o teste necessário para causar uma falha.

- O erro de compilação conta como uma **falha válida** para interromper o teste.

3ª Lei: Você escreve **apenas** o código suficiente para passar no teste.

- O programador implementa a **solução mínima** e **evita** funcionalidades extras.

O Ciclo de Desenvolvimento

As leis criam um ciclo que ocorre em questão de segundos.

- O programador alterna entre **falha**, **sucesso** e **limpeza** incessantemente.

As leis **bloqueiam** a criação de código legado desorganizado.

- O sistema mantém-se saudável pois o ciclo **impede** o acúmulo de **sujeira**.

Benefícios do TDD

1 - **Certeza:** A eliminação da dúvida sobre o código.

- O desenvolvedor **sabe** que o sistema funciona e pode implantá-lo **sem medo**.
- Testes manuais esporádicos deixam **lacunas** que geram **incerteza** na entrega.

2 - **Taxa de Injeção de Defeitos:** A redução drástica de bugs.

- O ciclo curto revela o erro segundos após ele ser digitado pelo programador.
- Mantém a taxa de erros **extremamente** baixa.

Benefícios do TDD

3 - Coragem: A confiança para alterar e limpar o código.

- Os testes **permitem** refatorar o sistema **sem** o **medo** de quebrar algo.
- Sem testes, a equipe **teme** a limpeza e o código degrada até **inviabilizar a manutenção** .

4 - Documentação: O manual técnico vivo e executável.

- Os testes unitários descrevem **exatamente** como o sistema se comporta na prática.
- Documentos em papel **desatualizam** , o teste **nunca mente** sobre o código.

Benefícios do TDD

5 - **Design**: A garantia de baixo acoplamento.

- A necessidade de testar primeiro **força** a criação de códigos modulares e pequenos.
- É **impossível** testar facilmente um código mal projetado ou muito acoplado.

6 - **Profissionalismo**: A ética de trabalho inegociável.

- O TDD aprimora a **certeza**, a **coragem**, a **documentação** e o **design** do projeto.
- A decisão de **não utilizar** a prática pode ser considerada **antiprofissional**.

O que o TDD NÃO é

O TDD **não atua** como religião ou **fórmula mágica**.

- O cumprimento mecânico das três leis não **garante** o sucesso automático.

A prática **não impede** totalmente a criação de código ruim.

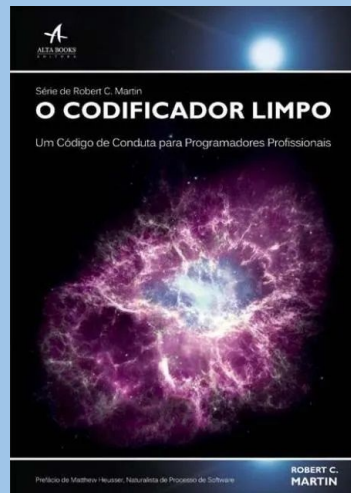
- O desenvolvedor descuidado ainda **pode** escrever testes **fracos ou ineficazes** .

Sobre o autor

Robert C. “Uncle Bob” Martin é desenvolvedor e consultor de software desde 1990. Ele é o fundador e o presidente da Object Mentor, Inc., uma equipe de consultores experientes que orientam seus clientes no mundo todo em C++, Java, C#, Ruby, OO, Padrões de Projeto, UML, Metodologias Agile e eXtreme Programming.

Este conjunto de slides foi elaborado a partir da obra:

MARTIN, Robert C. **O codificador limpo:**
Um código de conduta para programadores
profissionais. 1. ed. Rio de Janeiro: Alta
Books, 2012.



A equipe



Mariana Sônego, Autora

Aluna do primeiro período,
é bolsista do PET/ADS desde
novembro de 2025.



Leonardo Fattori, Revisor

Aluno do terceiro período, é
bolsista do PET/ADS desde
abril de 2025.



Lucas Oliveira, Tutor

Professor de Computação, é
tutor do PET/ADS desde
janeiro de 2023.